Propuesta de Gamificación: Sistema Interactivo Tamagotchi para SmartPlantBase

Asignatura: Arquitectura de Software · **Semestre:** 2026-2 · **Estudiante:** Esteban Castro · **Núcleo:** Arquitectura Hexagonal

1. Resumen y Concepto: La Matera Como Mascota Digital

En la Fase 2 de **SmartPlantBase** implementamos una arquitectura hexagonal completa que procesa telemetría de sensores (humedad del suelo en %, temperatura en °C y luminosidad en lux) y la evalúa contra rangos biológicos según la especie vegetal. Los datos se transmiten en tiempo real mediante WebSockets a un panel web.

Aunque el backend resuelve con solidez el diagnóstico biológico, para el usuario final las lecturas numéricas y las alertas pueden resultar frías y fáciles de ignorar. Esta propuesta plantea integrar una **mecánica de interacción tipo Tamagotchi**, convirtiendo a la planta en un organismo digital interactivo:

1. **Estado anímico visual:** En lugar de mostrar solo números, la planta reacciona con expresiones (emojis y caras ASCII) y mensajes en primera persona según su estado físico (contenta, sedienta, ahogada, sofocada, con frío).
2. **Progresión por buen cuidado (XP y rachas):** Mantener la planta en rangos saludables genera puntos de experiencia (*XP*) y acumula días de racha. Si el usuario descuida el riego o la expone a condiciones extremas, la planta decae visualmente y la racha se reinicia.
3. **Reacción inmediata a acciones reales:** Cuando el usuario riega la planta o la ubica en un lugar iluminado, el sensor detecta el cambio físico y la planta responde de inmediato con un estado de agradecimiento y animación en el frontend.
4. **Respeto estricto a la Arquitectura Hexagonal (RA3, RA4):** La gamificación se plantea como un subdominio independiente. La clase `ValidadorDeEstado` no se modifica ni se contamina con lógica de juego; el motor de gamificación consume el diagnóstico biológico de manera desacoplada a través de puertos y adaptadores dedicados.



2. Mecánica del Tamagotchi: Estados Anímicos y Progresión

2.1 Matriz de Estados Anímicos

El estado anímico se determina a partir del diagnóstico biológico emitido por el validador, los parámetros que presenten desviación y las transiciones rápidas de humedad (riego):

Estado Anímico	Emoji	Expresión ASCII	Disparador Biológico / Físico	Mensaje al Usuario	Impacto XP / Felicidad
RADIANTE	🌸✨	`( ˘ ³˘)♥`	Diagnóstico `SALUDABLE` + racha > 3 días consecutivos.	*"¡Estoy en mi mejor momento! El sol y el agua están perfectos."*	+25 XP / Felicidad 100%
CONTENTA	🌿😊	`(✿◠‿◠)`	Diagnóstico `SALUDABLE` (parámetros dentro de rango).	*"Me siento muy bien. Gracias por cuidar de mí."*	+10 XP / Felicidad 85%
SEDIENTA	💧🥺	`( •̀ ᴖ •́ )`	Humedad de sustrato en nivel `BAJO`.	*"Tengo sed; mi tierra está bastante seca. ¿Me das un poco de agua?"*	-5 XP / Felicidad 40%
AHOGADA	🌊🤢	`(×_×;)`	Humedad de sustrato en nivel `ALTO`.	*"¡Mucha agua! Mis raíces se están asfixiando, déjame secar un poco."*	-10 XP / Felicidad 30%
SOFOCADA	☀️🥵	`(;´д`)`	Temperatura o luz en nivel `ALTO`.	*"Hace demasiado calor aquí. ¿Podrías ponerme en un lugar fresco?"*	-10 XP / Felicidad 35%
CONGELADA	❄️🥶	`( >_< )`	Temperatura ambiental en nivel `BAJO`.	*"¡Qué frío! Mis hojas se están entumeciendo, protégeme del viento."*	-10 XP / Felicidad 35%
AGRADECIDA	💚🥰	`(つ✧ω✧)つ`	Incremento súbito de humedad (> 15% en menos de 5 min).	*"¡Agua fresca! Justo lo que necesitaba. ¡Muchas gracias!"*	+50 XP / Felicidad +30%
MARCHITA	🥀💀	`(x_x)`	Diagnóstico `CRITICO` (múltiples parámetros fuera de rango).	*"Mis hojas están decaídas... Necesito atención urgente."*	-30 XP / Felicidad 10%

2.2 Progresión por Niveles y Rachas

El sistema incentiva la constancia del usuario con una progresión sencilla dividida en 5 niveles:

- **Nivel 1: Semilla** (0 - 150 XP) — Avatar en etapa inicial, mensajes breves.
- **Nivel 2: Plántula** (151 - 450 XP) — Primeras hojas visibles, reacciones emocionales completas.
- **Nivel 3: Planta Joven** (451 - 1000 XP) — Follaje desarrollado, desbloquea consejos de cultivo.
- **Nivel 4: Planta Florecida** (1001 - 2000 XP) — Flores visibles, tolera un descuido leve sin perder racha.
- **Nivel 5: Guardián Botánico** (2000+ XP) — Planta completamente desarrollada, insignia de cuidador experto.

La racha aumenta en +1 cada 24 horas consecutivas en que la planta se mantenga en estado `SALUDABLE`. Si la planta entra en estado `CRITICO`, la racha se reinicia a 0 y la felicidad disminuye, pero la XP acumulada no baja de 0 para no frustrar al usuario.

2.3 Respuestas Adaptadas por Especie

Para darle realismo botánico, el tono de los mensajes varía según la especie configurada en la base de datos:

- **Sansevieria:** Resistente y sobria. Tolera la falta de agua y advierte con firmeza si la riegan en exceso: *"Recuerda que soy de clima seco; no acumules agua en mi plato"*.
- **Potos:** Expresiva y amante de la luz indirecta. Si pasa mucho tiempo en penumbra, pide acercarse a la ventana.
- **Suculenta:** Almacena agua en sus hojas. Entra en alerta rápida si detecta humedad saturada continua.
- **Helecho:** Exigente con la humedad ambiental y de suelo. Si el sustrato se seca, su mensaje es más urgente: *"¡Mis hojas se marchitan rápido! Rocíame un poco de agua"*.

---

3. Especificación de Requisitos

3.1 Requisitos Funcionales (RF)

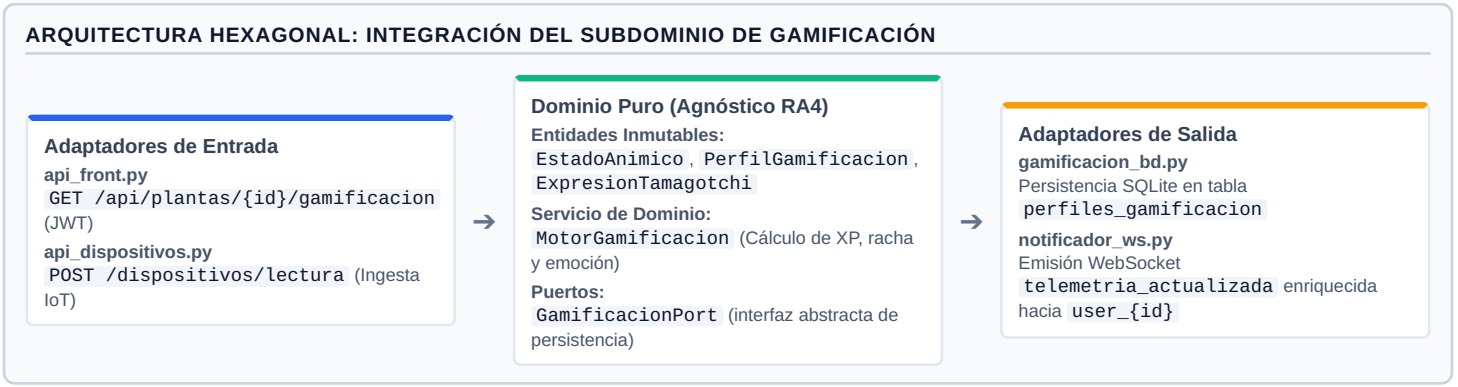
- **RF-01 (Cálculo de Ánimo):** El sistema debe derivar un estado anímico determinista a partir del `Diagnostico` biológico y la telemetría actual.
- **RF-02 (Cálculo de XP y Rachas):** Otorgar +10 XP por cada ciclo de lectura válida en estado `SALUDABLE`. Incrementar la racha de días tras 24 horas continuas sin alertas graves.
- **RF-03 (Detección de Riego):** Identificar incrementos de humedad superiores al 15% entre lecturas consecutivas para detonar el estado temporal `AGRADECIDA` con bonificación de +50 XP.
- **RF-04 (Persistencia del Perfil):** Almacenar de forma persistente la XP total, el nivel actual, los días de racha, el nivel de felicidad y el último estado anímico asociado a la planta.
- **RF-05 (Notificación WebSocket):** Integrar los datos del Tamagotchi dentro del evento WebSocket `telemetria_actualizada` emitido hacia la sala privada del usuario (`user_{id}`).
- **RF-06 (Consulta REST):** Ofrecer el endpoint `GET /api/plantas/{id}/gamificacion` protegido con JWT para consultar el resumen lúdico de una planta.

3.2 Requisitos No Funcionales (RNF)

- **RNF-01 (Agnosticismo de Dominio - RA4):** El módulo de gamificación debe desarrollarse en Python puro dentro de `matera/dominio/`, sin dependencias directas de Flask, SocketIO ni SQLite, permitiendo pruebas unitarias con dobles de prueba (`fakes.py`).
- **RNF-02 (Principio de Responsabilidad Única - SRP):** La clase `ValidadorDeEstado` mantiene únicamente la lógica botánica. La evaluación lúdica recae en un servicio separado (`MotorGamificacion`).
- **RNF-03 (Degradación Segura):** Un error en el cálculo o persistencia de la gamificación no debe interrumpir el registro de la telemetría ni la respuesta HTTP al dispositivo ESP32.
- **RNF-04 (Bajo Impacto de Rendimiento):** El procesamiento lúdico en memoria no debe superar los 5 milisegundos por lectura.
- **RNF-05 (Seguridad Multiusuario):** Los datos lúdicos están protegidos por el mismo control de acceso por usuario que el resto del sistema.

---

4. Impacto Arquitectónico y Alineación Hexagonal



4.1 Dominio (`matera/dominio/`)

- **`gamificacion.py`:** Define las entidades inmutables `EstadoAnimico`, `ExpresionTamagotchi` y `PerfilGamificacion`, además de la clase `MotorGamificacion`.
- **`puertos/salida.py`:** Incorpora la interfaz abstracta `GamificacionPort` con métodos `obtener_perfil(planta_id)` y `guardar_perfil(perfil)`.
- **`casos_uso/registrar_lectura.py`:** Se inyecta opcionalmente `GamificacionPort`. Tras ejecutar la validación biológica y registrar el historial, se invoca `MotorGamificacion` de forma protegida para emitir la telemetría enriquecida.

4.2 Adaptadores de Salida (`matera/adaptadores/salida/`)

- **`gamificacion_bd.py`:** Implementación en SQLite del puerto `GamificacionPort`, gestionando las operaciones CRUD sobre la tabla `perfiles_gamificacion`.
- **`notificador_ws.py`:** En el método `notificar_telemetria`, agrega la clave `"tamagotchi"` al diccionario emitido por WebSocket hacia la sala `user_{usuario_id}`.

4.3 Adaptadores de Entrada (`matera/adaptadores/entrada/`)

- **`api_front.py`:** Añade la ruta `GET /api/plantas/<planta_id>/gamificacion` para consultar el estado actual del Tamagotchi desde el frontend.

---

5. Diseño de Base de Datos y Persistencia

Para mantener la base de datos `data/matera.db` limpia y totalmente compatible con el esquema actual de la Fase 2, la información lúdica se almacena en una tabla dedicada mediante migración SQL. Cada registro en `perfiles_gamificacion` mantiene una relación 1:1 con `plantas`, eliminándose en cascada si la planta es eliminada:

```
-- data/bd/gamificacion.sql: Esquema relacional de gamificación
CREATE TABLE IF NOT EXISTS perfiles_gamificacion (
    planta_id TEXT PRIMARY KEY,
    xp_total INTEGER NOT NULL DEFAULT 0,
    nivel INTEGER NOT NULL DEFAULT 1,
    racha_dias INTEGER NOT NULL DEFAULT 0,
    felicidad INTEGER NOT NULL DEFAULT 100,
    ultimo_animo TEXT NOT NULL DEFAULT 'CONTENTA',
    ultima_actualizacion TEXT NOT NULL,
    FOREIGN KEY (planta_id) REFERENCES plantas(id) ON DELETE CASCADE
);

CREATE TABLE IF NOT EXISTS eventos_gamificacion (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    planta_id TEXT NOT NULL,
    tipo_evento TEXT NOT NULL, -- 'RIEGO', 'NIVEL_UP', 'RACHA_NUEVA'
    puntos_xp INTEGER NOT NULL,
    mensaje TEXT NOT NULL,
    marca_tiempo TEXT NOT NULL,
    FOREIGN KEY (planta_id) REFERENCES plantas(id) ON DELETE CASCADE
);

CREATE INDEX IF NOT EXISTS idx_eventos_planta ON eventos_gamificacion(planta_id, marca_tiempo DESC);
```

Este diseño no altera las tablas existentes (`plantas`, `usuarios`, `lecturas`), garantizando retrocompatibilidad y evitando migraciones destructivas.

---

6. Especificación de Contratos e Interfaces

6.1 Dominio: Entidades del Subdominio (`matera/dominio/gamificacion.py`)

Modelos inmutables en Python puro que representan el estado anímico, la expresión visual y el perfil acumulado del usuario con la planta:

```
from dataclasses import dataclass
from enum import Enum
from typing import Optional

class EstadoAnimico(str, Enum):
    RADIANTE = "RADIANTE"
    CONTENTA = "CONTENTA"
    SEDIENTA = "SEDIENTA"
    AHOGADA = "AHOGADA"
    SOFOCADA = "SOFOCADA"
    CONGELADA = "CONGELADA"
    AGRADECIDA = "AGRADECIDA"
    MARCHITA = "MARCHITA"

@dataclass(frozen=True)
class ExpresionTamagotchi:
    estado: EstadoAnimico
    emoji: str
    cara_ascii: str
    mensaje: str
    felicidad: int  # Rango 0 a 100

@dataclass(frozen=True)
class PerfilGamificacion:
    planta_id: str
    xp_total: int
    nivel: int
    racha_dias: int
    felicidad: int
    ultimo_animo: EstadoAnimico
    expresion: Optional[ExpresionTamagotchi] = None
```

6.2 Lógica del Motor de Gamificación (`MotorGamificacion`)

Servicio de dominio puro que procesa el diagnóstico botánico para derivar la emoción y calcular los puntos de experiencia:

```
class MotorGamificacion:
    """Reglas de negocio puras para la mecánica Tamagotchi."""

    @staticmethod
    def derivar_expresion(especie: str, diagnostico, es_riego: bool = False) -> ExpresionTamagotchi:
        if es_riego:
            return ExpresionTamagotchi(EstadoAnimico.AGRADECIDA, "💚🥰", "(つ✧ω✧)つ",
                                       "¡Agua fresca! Justo lo que necesitaba.", 100)
        if diagnostico.estado.value == "CRITICO":
            return ExpresionTamagotchi(EstadoAnimico.MARCHITA, "🥀💀", "(x_x)",
                                       "Mis hojas están decaídas, necesito atención urgente.", 15)
        for p in diagnostico.parametros:
            if p.nombre == "humedad" and p.estado.value == "BAJO":
                return ExpresionTamagotchi(EstadoAnimico.SEDIENTA, "💧🥺", "( •́ ︿ •̀ )",
                                           "Tengo sed; mi tierra está bastante seca.", 35)
            if p.nombre == "humedad" and p.estado.value == "ALTO":
                return ExpresionTamagotchi(EstadoAnimico.AHOGADA, "🌊🤢", "(x_x;)",
                                           "¡Demasiada agua en mi maceta!", 30)
            if p.nombre == "temperatura" and p.estado.value == "ALTO":
                return ExpresionTamagotchi(EstadoAnimico.SOFOCADA, "☀️🥵", "(;´д`)",
                                           "Hace un calor asfixiante aquí.", 40)
            if p.nombre == "temperatura" and p.estado.value == "BAJO":
                return ExpresionTamagotchi(EstadoAnimico.CONGELADA, "❄️🥶", "( >_< )",
                                           "¡Qué frío! Protégeme de las corrientes.", 40)
        return ExpresionTamagotchi(EstadoAnimico.CONTENTA, "🌿😊", "(✿◠‿◠)",
                                   "Todo está en armonía. Me siento radiante.", 95)

    @staticmethod
    def actualizar_perfil(perfil: PerfilGamificacion, exp: ExpresionTamagotchi, es_riego: bool) -> PerfilGamificacion:
        delta_xp = 50 if es_riego else (10 if exp.estado in (EstadoAnimico.CONTENTA, EstadoAnimico.RADIANTE) else 0)
        nueva_xp = perfil.xp_total + delta_xp
        nuevo_nivel = min(5, 1 + (nueva_xp // 250))
        nueva_felicidad = int((perfil.felicidad * 0.4) + (exp.felicidad * 0.6))
        return PerfilGamificacion(
            planta_id=perfil.planta_id, xp_total=nueva_xp, nivel=nuevo_nivel,
            racha_dias=perfil.racha_dias, felicidad=nueva_felicidad,
            ultimo_animo=exp.estado, expresion=exp
        )
```

6.3 Integración en el Caso de Uso `RegistrarLectura`

El caso de uso coordina la validación biológica y el subsistema de gamificación con aislamiento de fallos:

```
# matera/dominio/casos_uso/registrar_lectura.py
class RegistrarLectura(RegistrarLecturaPort):
    def registrar(self, dispositivo_id: str, valores: Mapping[str, float], marca_tiempo: str = "") -> Diagnostico:
        # 1. Validación biológica y persistencia del historial (núcleo incondicional)
        diagnostico = ValidadorDeEstado.evaluar(lectura, especie_entidad)
        self._historial.guardar_lectura(lectura, diagnostico)

        # 2. Gamificación desacoplada con degradación elegante
        datos_game = None
        if self._gamificacion is not None:
            try:
                perfil = self._gamificacion.obtener_perfil(planta.id)
                es_riego = self._detectar_riego(valores, planta.id)
                exp = MotorGamificacion.derivar_expresion(planta.especie, diagnostico, es_riego)
                perfil = MotorGamificacion.actualizar_perfil(perfil, exp, es_riego)
                self._gamificacion.guardar_perfil(perfil)
                datos_game = perfil
            except Exception as err:
                logger.warning("Fallo no fatal en subsistema de gamificación: %s", err)

        # 3. Notificación WebSocket con telemetría + bloque tamagotchi
        if self._notificador is not None:
            self._notificador.notificar_telemetria(planta, lectura, diagnostico, datos_game)
        return diagnostico
```

6.4 Enriquecimiento del Payload WebSocket

El evento existente `telemetria_actualizada` añade un bloque `"tamagotchi"` para que el frontend actualice el avatar sin peticiones HTTP adicionales:

```
{
  "planta_id": "planta-01",
  "planta_nombre": "Mi Helecho",
  "dispositivo_id": "esp32-matera-01",
  "especie": "helecho",
  "valores": { "humedad": 38.5, "temperatura": 21.0, "luz": 450.0 },
  "estado": "EN_RIESGO",
  "tamagotchi": {
    "animo": "SEDIENTA",
    "emoji": "💧🥺",
    "cara_ascii": "( •̀ ᗣ •́ )",
    "mensaje": "Tengo sed; mi tierra está bastante seca.",
    "felicidad": 35,
    "xp_total": 280,
    "nivel": 2,
    "racha_dias": 4
  }
}
```

---

7. Diseño de Interfaz en el Frontend

Siguiendo la restricción **RA2** de la materia (Vanilla JavaScript, HTML5 y CSS3 sin frameworks pesados), la interfaz del Tamagotchi se implementa de manera ligera:

7.1 Componente HTML en `#live-panel` (`frontend/index.html`)

```
<!-- Widget Tamagotchi integrado en el panel en vivo -->
<div class="card tamagotchi-card" id="tamagotchi-widget">
    <div class="tamagotchi-avatar-container">
        <div class="tamagotchi-avatar" id="gotchi-avatar">🌿</div>
        <div class="tamagotchi-face" id="gotchi-face">(✿◠‿◠)</div>
    </div>
    <div class="tamagotchi-dialog">
        <div class="tamagotchi-bubble">
            <span class="bubble-tag" id="gotchi-mood-tag">CONTENTA</span>
            <p id="gotchi-message">"Todo está en orden. ¡Gracias por cuidar de mí!"</p>
        </div>
        <div class="tamagotchi-stats">
            <div class="stat-bar-group">
                <span class="stat-label">Felicidad: <strong id="gotchi-happy-val">95%</strong></span>
                <div class="progress-track"><div class="progress-fill fill-happy" id="gotchi-happy-bar" style="width: 95%;"></
div></div>
            </div>
            <div class="stat-bar-group">
                <span class="stat-label">Nivel: <strong id="gotchi-level-val">Nivel 2 (Plántula)</strong> · XP: <span id="gotchi-
xp-val">280</span></span>
                <div class="progress-track"><div class="progress-fill fill-xp" id="gotchi-xp-bar" style="width: 56%;"></div></div>
            </div>
        </div>
    </div>
</div>
```

7.2 Animaciones CSS Reactivas (`frontend/css/style.css`)

Las animaciones permiten que la planta transmita visualmente su estado mediante movimientos suaves:

```
.tamagotchi-avatar {
    font-size: 3rem;
    animation: breathing 3s ease-in-out infinite;
}
@keyframes breathing {
    0%, 100% { transform: translateY(0) scale(1); }
    50% { transform: translateY(-3px) scale(1.04); }
}
.mood-SEDIENTA .tamagotchi-avatar, .mood-MARCHITA .tamagotchi-avatar {
    animation: droop 2s ease-in-out infinite;
    filter: grayscale(35%);
}
@keyframes droop {
    0%, 100% { transform: rotate(0deg); }
    50% { transform: rotate(-6deg) translateY(2px); }
}
.mood-AGRADECIDA .tamagotchi-avatar, .mood-RADIANTE .tamagotchi-avatar {
    animation: bounce 0.8s ease-in-out infinite alternate;
}
@keyframes bounce {
    0% { transform: scale(1); }
    100% { transform: scale(1.12) translateY(-4px); }
}
```

7.3 Actualización Reactiva en `frontend/js/ui.js`

Al recibir el evento WebSocket, `PlantUI.renderizarTamagotchi(data.tamagotchi)` actualiza el DOM de forma síncrona sin recargar la página:

```
renderizarTamagotchi(t) {
    if (!t) return;
    document.getElementById("gotchi-avatar").textContent = t.emoji;
    document.getElementById("gotchi-face").textContent = t.cara_ascii;
    document.getElementById("gotchi-mood-tag").textContent = t.animo;
    document.getElementById("gotchi-message").textContent = `"${t.mensaje}"`;
    document.getElementById("gotchi-happy-val").textContent = `${t.felicidad}%`;
    document.getElementById("gotchi-happy-bar").style.width = `${t.felicidad}%`;
    document.getElementById("gotchi-xp-val").textContent = t.xp_total;
    document.getElementById("gotchi-level-val").textContent = `Nivel ${t.nivel}`;
    const widget = document.getElementById("tamagotchi-widget");
    widget.className = `card tamagotchi-card mood-${t.animo}`;
}
```

---

8. Plan de Implementación (Roadmap de 5 Hitos)

Hito	Alcance y Tareas Principales	Archivos Involucrados	Criterio de Verificación
Hito 1	**Dominio de Gamificación Puro** Crear entidades `EstadoAnimico`, `PerfilGamificacion` y servicio `MotorGamificacion`. Pruebas unitarias aisladas sin base de datos ni servidor.	`matera/dominio/gamificacion.py` `tests/test_gamificacion.py`	100% de pruebas pasando con `pytest` en < 0.1s. Cobertura de todas las transiciones de estado anímico.

Hito 2	**Puertos y Persistencia SQLite**
	Definir interfaz `GamificacionPort`, adaptador `gamificacion_bd.py` y script de migración SQL.
		`matera/dominio/puertos/salida.py`
		`matera/adaptadores/salida/gamificacion_bd.py`
		`data/bd/gamificacion.sql`
			Pruebas de integración verificando creación de perfil, acumulación de XP y recuperación en SQLite.

Hito 3	**Integración en Casos de Uso y WebSockets** Conectar `RegistrarLectura` con `MotorGamificacion` y enriquecer `SocketIONotificadorAdapter`.	`matera/dominio/casos_uso/registrar_lectura.py` `matera/adaptadores/salida/notificador_ws.py`	Al enviar telemetría desde el simulador ESP32, el evento WebSocket contiene el bloque `tamagotchi`.

Hito 4	**Frontend y Widget Interactivo**
	Maquetar el widget Tamagotchi en HTML, añadir animaciones CSS y binding reactivo en JavaScript.
		`frontend/index.html`
		`frontend/css/style.css`
		`frontend/js/ui.js`
			En el navegador, el avatar cambia de cara, color y animación al llegar nuevas lecturas sin refrescar la página.

Hito 5	**Simulación de Riego y Verificación E2E** Agregar bandera `--regar` en `simulador_esp32.py` para simular un salto de humedad y verificar la reacción.	`simulador_esp32.py` `tests/test_e2e_gamificacion.py`	El simulador detona el estado `AGRADECIDA`, otorgando +50 XP y actualizando la interfaz en tiempo real.

---

9. Conclusiones y Próximos Pasos

1. **Mayor conexión con el usuario sin alterar el núcleo técnico:** La gamificación transforma los datos crudos de sensores en una experiencia interactiva intuitiva, aumentando el compromiso del usuario con el cuidado de su planta sin perjudicar la seriedad del diagnóstico.
2. **Cumplimiento estricto de la Arquitectura Hexagonal:** El subdominio de gamificación respeta el aislamiento del dominio (RA4) y el principio de responsabilidad única (SRP). El validador biológico permanece intacto y la nueva funcionalidad se integra a través de puertos y adaptadores.
3. **Viabilidad técnica inmediata:** La solución aprovecha la infraestructura ya probada en la Fase 2 (SQLite, WebSockets por sala privada, frontend Vanilla JS), por lo que su desarrollo puede ejecutarse de manera incremental en los 5 hitos propuestos sin riesgo de regresión sobre los 106 tests actuales.